



roadmap.md — Orvio Execution Roadmap

Version: 0.1 (MVP-ready)

Last updated: 2026-09-29

Owner: Engineering (Code X)

Applies to: Product, Engineering, QA, DevOps, AI coding agents

0. How to Use This Document

This is the **execution source of truth** for Orvio. It translates PRD, architecture, and security requirements into an ordered, testable plan.

Chain of truth:

PRD → architecture.md → database.md → api-contract.md → auth.md → security.md → testing.md → roadmap.md → AGENTS.md → Implementation

Rules:

- Do not override higher-level requirements.
- Do not jump phases without justification.
- AI agents must read this before implementing features.

1. Roadmap Purpose

Translate product and technical requirements into an **ordered execution plan** that connects:

```
Product Vision
  ↓
PRD
  ↓
Architecture
  ↓
Technical Foundations
  ↓
Features
  ↓
Testing
  ↓
Security
  ↓
Release
```

↓
Post-release Iteration

Goal: Build foundations first, then critical user value, with security and testing integrated.

2. Source-of-Truth Hierarchy

```
PRD.md
  ↓
architecture.md
  ↓
database.md
  ↓
api-contract.md
  ↓
auth.md
  ↓
security.md
  ↓
testing.md
  ↓
roadmap.md
  ↓
AGENTS.md
  ↓
Implementation
```

Conflict handling: Roadmap never overrides confirmed product/arch/security requirements. If conflict arises, identify and escalate.

3. Roadmap Principles

- **Foundations first:** Auth, DB, API before protected features.
- **Critical user value:** MVP workflows prioritized.
- **Minimize dependencies:** Avoid premature abstraction/microservices.
- **Security foundational:** Integrated per phase, not bolted on.
- **Testing part of done:** No feature complete without tests.
- **Vertical slices:** End-to-end thin slices where possible.
- **Incremental releases:** Keep app runnable throughout.
- **Explicit dependencies:** Document hard/soft deps.
- **Backward compatibility:** Preserve API contracts.
- **Measurable completion:** Every item has exit criteria.

4. Current State

CONFIRMED:

- Project context from PRD, architecture, database, API, auth, security, testing docs.
- Stack: React + Vite + TS | Node.js + Fastify + TS | PostgreSQL + Prisma.
- Multi-tenant SaaS (org/branch model).
- MVP: Inventory + POS + WhatsApp Orders + Payments (Paystack).

CURRENT STATE (assumed from context):

- Repository structure: apps/web, apps/api, packages/shared.
- No production deployment yet.

OPEN DECISION:

- Exact CI/CD provider (GitHub Actions assumed).
- Hosting provider (PaaS assumed).
- Email provider (deferred).

RECOMMENDATION:

- Use GitHub Actions, Render/Railway/Fly for PaaS.

5. Product Vision

Vision: Enable Nigerian SMBs to sell everywhere and track everything with a simple, affordable operations platform.

Problem: Stockouts, theft, manual spreadsheets; tools too heavy (Odoo) or narrow (payment-only POS).

Target users: Shop owners, managers, cashiers, storekeepers.

Core value: Real-time stock, offline POS, WhatsApp orders, local payments.

Long-term: Expand to purchasing, eCommerce, light manufacturing.

6. Roadmap Outcomes

- **Product foundation:** Repo, tooling, CI/CD.
- **Auth foundation:** Login, tokens, sessions.
- **Core platform:** Org/branch, products, stock.
- **Core workflows:** POS, WhatsApp orders, payments.
- **Security foundation:** RBAC, tenant isolation, input validation.
- **MVP validation:** 50–100 shops in Lagos.

- **Beta/production readiness:** Hardening, monitoring.
- **Launch:** Public release.
- **Post-launch:** Purchasing, advanced reports.

7. Phase Model

Phase 0 — Discovery / Validation

- **Objective:** Validate problem, MVP scope.
- **Scope:** PRD, architecture decisions.
- **Exit:** PRD + architecture.md approved.

Phase 1 — Foundation

- **Objective:** Repo, tooling, DB, auth skeleton.
- **Scope:** Monorepo, Fastify, Prisma, JWT.
- **Exit:** Can register/login, create org.

Phase 2 — Core Platform

- **Objective:** Org/branch, products, stock.
- **Scope:** Inventory module, RBAC.
- **Exit:** Can manage products/stock.

Phase 3 — Core Workflows

- **Objective:** POS, WhatsApp orders, payments.
- **Scope:** POS checkout, WA orders, Paystack.
- **Exit:** Can sell via POS/WA, collect payments.

Phase 4 — Security / Hardening

- **Objective:** Tenant isolation, rate limiting, audit.
- **Scope:** RBAC enforcement, logs, monitoring.
- **Exit:** Security tests pass.

Phase 5 — MVP Validation

- **Objective:** Pilot with 10–20 shops.
- **Scope:** Bug fixes, UX tweaks.
- **Exit:** 80% 30-day retention.

Phase 6 — Beta / Production Readiness

- **Objective:** Scale prep, backups, alerts.
- **Scope:** Observability, DR.
- **Exit:** Ready for public launch.

Phase 7 — Launch

- **Objective:** Public release.
- **Scope:** Marketing, onboarding.
- **Exit:** 100 paying shops.

Phase 8 — Post-Launch Expansion

- **Objective:** Purchasing, reports, integrations.
- **Scope:** Phase 2+ features.
- **Exit:** Roadmap v2.

8. MVP Definition

MUST HAVE:

- Auth (email/password, JWT).
- Org/branch, products, stock.
- POS (offline-capable).
- WhatsApp orders + Paystack.
- RBAC (Owner, Manager, Cashier, Storekeeper).
- Basic reports (sales, stock).

SHOULD HAVE:

- Email verification (defer to Phase 2).
- Advanced reports (Phase 2).

COULD HAVE:

- OAuth (Phase 3).
- MFA (Phase 3).

POST-MVP:

- Purchasing, eCommerce, logistics.

FUTURE:

- Light manufacturing, multi-currency.

9. MVP Boundary

Deliberately deferred:

- Email verification (UX complexity).
- OAuth/MFA (later).
- Purchasing (Phase 2).
- eCommerce (Phase 3).
- Advanced reports (Phase 2).
- Logistics integrations (Phase 3).

Reason: Focus on core inventory + POS + WA + payments.

10. Feature Inventory

Category	Feature	Phase	Priority	Dependencies	Status
Platform	Monorepo setup	1	High	None	NOT STARTED
Auth	JWT login/signup	1	High	None	NOT STARTED
Org	Org/branch CRUD	2	High	Auth	NOT STARTED
Inventory	Products CRUD	2	High	Org	NOT STARTED
Inventory	Stock adjustments	2	High	Products	NOT STARTED
POS	POS checkout	3	High	Inventory	NOT STARTED
Orders	WhatsApp orders	3	High	Payments	NOT STARTED
Payments	Paystack integration	3	High	Orders	NOT STARTED
Reports	Sales/stock reports	3	Medium	POS, Inventory	NOT STARTED
Security	RBAC enforcement	2	High	Auth	NOT STARTED
Security	Tenant isolation	2	High	Org	NOT STARTED
Infra	CI/CD	1	High	None	NOT STARTED
Infra	Monitoring (Sentry)	4	Medium	Auth	NOT STARTED

11. Feature Prioritization

Criteria:

- User value (SMB pain points).
- Business value (retention, revenue).
- Technical dependency (auth → org → inventory).
- Security risk (tenant isolation critical).

- Complexity (start simple).

Priority order:

1. Auth foundation.
2. Org/branch + RBAC.
3. Inventory (products, stock).
4. POS + WhatsApp + Payments.
5. Reports + hardening.

12. Dependency Graph

```
Repo/Tooling
  ↓
Auth (JWT)
  ↓
Org/Branch + RBAC
  ↓
Inventory (Products, Stock)
  ↓
POS + WhatsApp Orders
  ↓
Payments (Paystack)
  ↓
Reports
  ↓
Security Hardening
  ↓
MVP Pilot
  ↓
Launch
```

Hard dependencies: Auth before org; org before inventory; inventory before POS.

Parallelizable: Frontend shell + backend auth.

Blocking: DB schema before API.

External: Paystack sandbox, WhatsApp BSP.

13. Critical Path

Critical path:

1. Auth foundation (blocks all protected features).
2. Org/branch + RBAC (blocks inventory).
3. Inventory schema (blocks POS/WA).
4. POS checkout (core workflow).

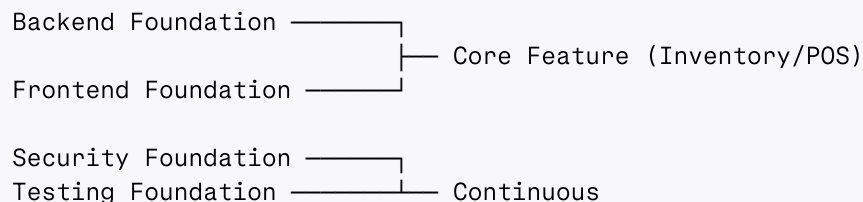
5. Payments (revenue).

Highest risk: Auth/security bugs, tenant isolation failures.

Parallel work: Frontend onboarding while backend builds inventory API.

Delayed: Advanced reports, purchasing.

14. Parallel Workstreams



Dependencies: API contract must stabilize before frontend depends heavily.

15. Technical Foundation Roadmap

Sequence:

1. Monorepo (pnpm, TS config).
2. Frontend (Vite, React, TanStack Query).
3. Backend (Fastify, Prisma).
4. DB (PostgreSQL, migrations).
5. API (REST, validation).
6. Auth (JWT, refresh).
7. Env config (dev/staging/prod).
8. Logging (Pino), error handling.
9. Testing (Vitest, Playwright).
10. Security (CORS, rate limit).
11. CI/CD (GitHub Actions).
12. Deployment (PaaS).
13. Monitoring (Sentry).

16. Database Roadmap

Sequence:

1. Initial schema (users, orgs, branches).
2. Core entities (products, stock_items, stock_movements).

3. Relationships (FKs, uniques).
4. Constraints (no negative stock).
5. Indexes (org_id, branch_id).
6. Seed data (test fixtures).
7. Migrations (Prisma).
8. Data access layer (repositories).
9. Tenant isolation (org_id scoping).
10. Audit logging (audit_logs).
11. Reporting views (future).

17. API Roadmap

Sequence:

1. API foundation (Fastify, validation).
2. Auth endpoints (/auth/*).
3. Org endpoints (/orgs/*).
4. Inventory endpoints (/products, /stock/*).
5. POS endpoints (/pos/sales).
6. Orders endpoints (/orders/whatsapp).
7. Payments endpoints (/payments/*).
8. Reports endpoints (/reports/*).
9. Error handling, pagination, filtering.
10. Webhooks (/webhooks/paystack, /webhooks/whatsapp).
11. Documentation (OpenAPI future).

18. Authentication Roadmap

Sequence:

1. Identity (users table).
2. Signup (/auth/register).
3. Login (/auth/login).
4. Sessions/tokens (JWT access + refresh).
5. Logout (/auth/logout).
6. Password reset (/auth/reset-password).
7. Email verification (Phase 2).
8. OAuth (Phase 3).

9. MFA (Phase 3).
10. Session revocation (future).

19. Authorization Roadmap

Sequence:

1. Roles (owner, manager, cashier, storekeeper).
2. Permissions (middleware).
3. Resource ownership (org_id).
4. Tenant scope (org/branch).
5. Admin scope (internal admin).
6. Custom roles (future).
7. Frontend permission handling (UX).
8. Backend enforcement (RBAC).
9. Tests (IDOR, tenant isolation).

20. Security Roadmap

Sequence:

1. Auth (JWT, hashing).
2. AuthZ (RBAC, tenant isolation).
3. Input validation (Zod).
4. Output encoding (React/JSON).
5. Secure sessions (httpOnly cookies).
6. CORS/CSRF.
7. Rate limiting.
8. Security headers (CSP, HSTS).
9. Secrets management (env vars).
10. Audit logging.
11. File security (signed URLs).
12. Webhook security (HMAC).
13. Dependency security (Snyk).
14. Security testing (SAST/DAST future).

21. Testing Roadmap

Sequence:

1. Testing infra (Vitest, Playwright).
2. Unit tests (domain, validation).
3. API/integration tests (API + DB).
4. Security tests (authZ, IDOR).
5. Component tests (React).
6. Critical E2E (signup → sale).
7. Regression suite.
8. Production smoke tests.

22. Frontend Roadmap

Sequence:

1. App shell (routing, layouts).
2. Auth UI (login/signup).
3. Onboarding (org/branch setup).
4. Inventory UI (products, stock).
5. POS UI (cart, checkout).
6. Orders UI (WhatsApp orders).
7. Reports UI (sales, stock).
8. Settings UI (users, roles).
9. Loading/empty/error states.
10. Permission-aware UI.
11. Responsive behavior.
12. Accessibility (keyboard, aria).

23. Backend Roadmap

Sequence:

1. Server foundation (Fastify).
2. Config (env, constants).
3. Validation (Zod).
4. Error handling (centralized).
5. Auth (JWT middleware).

6. AuthZ (RBAC middleware).
7. Services (business logic).
8. Repositories (Prisma).
9. API routes.
10. Transactions.
11. Background jobs (webhooks).
12. Webhooks (Paystack, WA).
13. Integrations (Paystack SDK).
14. Logging (Pino).
15. Observability (Sentry).

24. UI/UX Roadmap

Sequence:

1. Design system (colors, typography).
2. Navigation (sidebar, breadcrumbs).
3. Core screens (dashboard, inventory, POS).
4. Forms (validation, errors).
5. Empty states (CTAs).
6. Loading states (skeleton/spinner).
7. Error states (retry).
8. Confirmation flows (destructive actions).
9. Accessibility (focus, labels).
10. Responsive behavior (mobile POS).

25. Integration Roadmap

Integration	Purpose	MVP	Dependencies	Security
Paystack	Payments	Yes	Orders	HMAC webhooks
WhatsApp BSP	Orders	Yes	Products	Auth, templates
Email (future)	Verification	No	Auth	NDPA compliance
OAuth (future)	Social login	No	Auth	OAuth security

26. Notification / Event Roadmap

Sequence:

1. In-app notifications (toasts).
2. Email (Phase 2, verification).
3. SMS (Phase 3, OTP).
4. Push (future).
5. Domain events (internal).
6. Webhooks (Paystack, WA).
7. Background processing (queues future).

27. Search / Reporting / Analytics Roadmap

MVP:

- Search (products by name/SKU).
- Filtering (branch, category).
- Reports (sales, stock).
- Export (CSV).

Post-MVP:

- Advanced reports (ABC analysis).
- Dashboards (KPIs).
- Analytics (cohort, retention).

28. Billing / Payments Roadmap

Sequence:

1. Plans (Free, Starter, Growth).
2. Pricing (₦5k-₦25k/month).
3. Checkout (Paystack).
4. Subscription state (subscriptions table).
5. Webhooks (renewal/failure).
6. Failed payments (grace period).
7. Refunds (future).
8. Invoices (future).
9. Entitlements (feature gating).
10. Billing portal (future).

29. Administration Roadmap

Sequence:

1. Org admin (users, roles).
2. User admin (suspend, reset).
3. Role management (RBAC).
4. Permission management.
5. Audit logs (view).
6. Configuration (tax, currency).
7. Feature flags (future).
8. Support tooling (impersonation future).

30. Observability Roadmap

Sequence:

1. Logging (Pino, structured).
2. Error tracking (Sentry).
3. Metrics (latency, errors).
4. Health checks (/health).
5. Request IDs (correlation).
6. Audit logs (security events).
7. Performance monitoring (APM future).
8. Alerts (PagerDuty future).

31. DevOps / Infrastructure Roadmap

Sequence:

1. Dev environment (local Postgres).
2. Test environment (ephemeral DB).
3. Staging (PaaS).
4. Production (PaaS).
5. CI (GitHub Actions).
6. CD (auto-deploy).
7. Env vars (secrets).
8. DB migrations (Prisma).

9. Backups (daily).
10. Monitoring (Sentry, logs).
11. Deployment (staging → prod).
12. Rollback (revert).
13. DR (restore tests).

32. Release Strategy

Lifecycle:

```
Development
  ↓
Internal Testing
  ↓
Alpha (team)
  ↓
Private Beta (10-20 shops)
  ↓
Public Beta (100 shops)
  ↓
Production
```

Entry criteria: Tests pass, security review.

Exit criteria: Retention targets, bug thresholds.

Testing: E2E, smoke, security.

Monitoring: Errors, latency.

Rollback: Revert deploy.

33. Release Gates

Hard gates:

- [] Required tests pass.
- [] Critical security tests pass.
- [] Auth works.
- [] AuthZ works.
- [] Tenant isolation verified.
- [] DB migrations verified.
- [] API contract verified.
- [] Critical E2E flows pass.

- ☐ No known blocking regression.
- ☐ Env config exists.
- ☐ Monitoring exists.
- ☐ Rollback path exists.

34. Definition of Done

A feature is complete when:

- ☐ Requirement implemented.
- ☐ API implemented.
- ☐ DB changes completed.
- ☐ Validation implemented.
- ☐ AuthZ implemented.
- ☐ Security reviewed.
- ☐ Tests added.
- ☐ Regression coverage added.
- ☐ Loading/empty/error states implemented.
- ☐ Documentation updated.
- ☐ Observability considered.
- ☐ Code reviewed.

35. Milestone Structure

Milestone 1 — Foundation

Objective: Repo, tooling, auth skeleton.

Why: Enables all subsequent work.

Scope: Monorepo, Fastify, Prisma, JWT.

Dependencies: None.

Deliverables: Can register/login, create org.

Tests: Unit (auth), integration (API+DB).

Security: Password hashing, JWT validation.

Risks: None.

Exit Criteria: Auth flow works end-to-end.

36. Epics and Work Packages

Epic: Auth Foundation

- Backend: JWT, hashing.
- DB: `users` table.
- API: `/auth/*`.
- Frontend: Login/signup.
- Security: Rate limiting.
- Testing: Auth tests.

Epic: Inventory Core

- Backend: Products, stock services.
- DB: `products`, `stock_items`.
- API: `/products`, `/stock/*`.
- Frontend: Inventory UI.
- Security: Tenant isolation.
- Testing: Stock adjustment tests.

37. Task Decomposition

Task: Implement JWT login

- **Objective:** Authenticate user, issue tokens.
- **Dependencies:** `users` table, hashing.
- **Output:** `/auth/login` endpoint, frontend form.
- **Tests:** Valid/invalid credentials, rate limit.
- **Acceptance:** Can login, access token works.

38. AI Coding Agent Execution

AI agents must:

- Read current phase/milestone.
- Read PRD/arch/db/api/auth/security/testing.
- Inspect repo for existing code.
- Identify dependencies.
- Plan smallest change.
- Implement, test, document.
- Report completion accurately.

AI agents must not:

- Jump phases.
- Implement deferred features.
- Expand scope.
- Invent requirements.
- Skip security/tests.

39. Roadmap Drift Prevention

When implementation differs:

```
Observed Difference
  ↓
Determine Why
  ↓
Compare Against Source of Truth
  ↓
Update Correct Artifact (roadmap or impl)
```

Do not blindly change roadmap to match incorrect impl.

40. Change Management

Changes due to:

- New product info.
- User feedback.
- Technical/security discovery.
- Dependency changes.
- Regulatory requirements.

Document:

- Original plan.
- Reason for change.
- Impact.
- New plan.
- Dependencies.
- Decision.

41. Technical Debt Roadmap

Debt:

- Security debt (missing controls).
- Architecture debt (tight coupling).
- Testing debt (low coverage).
- Performance debt (slow queries).
- Maintainability debt (no docs).
- Infrastructure debt (no backups).

Prioritize: Security > data loss > performance.

42. Risk Register

Risk	Probability	Impact	Phase	Mitigation	Owner
Auth bug	Medium	High	1	Tests, review	Backend
Tenant leakage	Low	Critical	2	Isolation tests	Backend
Paystack downtime	Medium	High	3	Queue, retry	Backend
Scope creep	High	Medium	All	MVP boundary	PM
Flaky tests	Medium	Medium	All	Quarantine, fix	QA

43. Dependency Register

Dependency	Why	Blocks	Status	Mitigation
PostgreSQL	Data store	All	Available	Use managed DB
Paystack	Payments	Revenue	Available	Sandbox testing
WhatsApp BSP	Orders	WA flow	Available	Mock in CI
GitHub Actions	CI/CD	Deploy	Available	Fallback manual

44. Open Decisions

- **Launch date:** OPEN DECISION.
- **Email provider:** OPEN DECISION (Phase 2).
- **OAuth provider:** OPEN DECISION (Phase 3).
- **Performance thresholds:** OPEN DECISION (baseline later).
- **Browser support:** OPEN DECISION (Chrome, Firefox assumed).

45. Recommendations

- **CI/CD:** GitHub Actions.
- **Hosting:** Render/Railway/Fly.
- **Monitoring:** Sentry + Pino.
- **Testing:** Vitest + Playwright.
- **Load testing:** k6 (Phase 4).

46. Do Not Build Yet

- Email verification (Phase 2).
- OAuth/MFA (Phase 3).
- Purchasing (Phase 2).
- eCommerce (Phase 3).
- Advanced reports (Phase 2).
- Logistics integrations (Phase 3).
- Microservices (future).
- Redis (Phase 2+).
- Elasticsearch (future).

47. Roadmap Status System

Status labels:

- NOT STARTED
- PLANNED
- IN PROGRESS
- BLOCKED
- PARTIALLY COMPLETE
- COMPLETE
- DEFERRED
- CANCELLED

48. Roadmap Table

Phase	Milestone	Objective	Dependencies	Status	Exit Criteria
1	Foundation	Repo, auth skeleton	None	NOT STARTED	Can register/login

Phase	Milestone	Objective	Dependencies	Status	Exit Criteria
2	Core Platform	Org/branch, inventory	Auth	NOT STARTED	Can manage products/stock
3	Core Workflows	POS, WA, payments	Inventory	NOT STARTED	Can sell, collect payments
4	Security/Hardening	RBAC, monitoring	Core	NOT STARTED	Security tests pass
5	MVP Validation	Pilot 10–20 shops	Security	NOT STARTED	80% retention
6	Beta/Prod Ready	Backups, alerts	MVP	NOT STARTED	Ready for launch
7	Launch	Public release	Beta	NOT STARTED	100 paying shops
8	Post-Launch	Purchasing, reports	Launch	NOT STARTED	Roadmap v2

49. Now / Next / Later

Now:

- Phase 1 (Foundation): Monorepo, auth, CI/CD.

Next:

- Phase 2 (Core Platform): Org/branch, inventory.

Later:

- Phase 3+ (POS, WA, payments, hardening, launch).

50. Current Next Action

Objective: Implement Phase 1 (Foundation).

Source docs: `architecture.md`, `auth.md`, `database.md`, `api-contract.md`.

Dependencies: None.

Deliverables:

- Monorepo setup.
- Fastify server.
- Prisma schema (users, orgs).
- JWT login/signup.
- CI/CD pipeline.

Tests:

- Unit (auth logic).
- Integration (API + DB).

Security:

- Password hashing, JWT validation.

Definition of Done:

- Can register/login, create org.
- Tests pass.
- CI green.

51. Roadmap Traceability

```
PRD Requirement (PRD-001)
  ↓
Roadmap Milestone (Milestone 2)
  ↓
Feature (Inventory Core)
  ↓
Architecture (Modular Monolith)
  ↓
API (/products, /stock/*)
  ↓
Implementation (services, repos)
  ↓
Tests (unit, integration)
  ↓
Release (Phase 2)
```

52. Success Metrics

OPEN DECISION:

- Activation rate.
- 30/90-day retention.
- MRR.
- Error rates.
- Performance (p95 latency).

53. Post-Launch Roadmap

Sequence:

1. Launch stabilization (bug fixes).
2. User feedback (prioritize).
3. Performance improvements.
4. Product expansion (purchasing).
5. Advanced features (eCommerce).

54. Roadmap Review Cadence

Review when:

- Major milestone completes.
- Requirements change.
- Architecture changes.
- Technical/security discovery.
- User feedback changes priorities.
- Launch assumptions change.

Do not rewrite constantly.

55. AI Agent Roadmap Safety Rules

AI agents must:

- Determine current phase/milestone.
- Read source docs.
- Inspect repo.
- Identify dependencies.
- Plan smallest change.
- Test, document, report.

AI agents must not:

- Jump phases.
- Implement deferred features.
- Expand scope.
- Skip security/tests.

56. Roadmap Invariants

- Security work cannot be bypassed.
- Feature not complete without tests.
- Critical features need test coverage.
- Protected resources need authZ.
- DB changes follow migrations.
- API respects `api-contract.md`.
- Auth respects `auth.md`.
- Security respects `security.md`.
- Testing respects `testing.md`.
- Future features don't enter current milestone.
- AI agents don't expand scope.

57. Final Roadmap Quality Review

- ☐ Reflects PRD.
- ☐ MVP scope clear.
- ☐ Post-MVP separated.
- ☐ Dependencies explicit.
- ☐ Critical path identified.
- ☐ Parallel work identified.
- ☐ Foundations before features.
- ☐ Security integrated.
- ☐ Testing integrated.
- ☐ Auth before protected workflows.
- ☐ AuthZ foundational.
- ☐ DB/API sequenced.
- ☐ Frontend/backend deps clear.
- ☐ Release gates exist.
- ☐ DoD exists.
- ☐ Current state documented.
- ☐ Next action identified.
- ☐ Deferred work explicit.
- ☐ Technical debt tracked.
- ☐ Risks documented.

- ☐ Open decisions separated.
- ☐ Recommendations labeled.
- ☐ AI rules explicit.
- ☐ No facts invented.
- ☐ No unnecessary tech/scope.
- ☐ Practical and executable.

End of roadmap.md